



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

Department of Computer Science and Engineering

2025-2026

Odd Semester

DATA STRUCTURES AND ALGORITHMS-3

(25CS2103E)

TREE DIAMETER COMPUTATION & VISUALIZATION SYSTEM

Pranitha M. 2520030365

Angela Jose 2520030365

I. Harshika 2520030607

COURSE INSTRUCTOR

Dr. J Sirisha Devi

Professor

Department of Computer Science and Engineering

Tree Diameter Computation and Visualization System

Project Overview

The Tree Diameter Computation and Visualization System is an interactive application that finds the longest path between any two nodes of a tree and presents the result visually. The project demonstrates tree data structures, tree traversal, efficient path computation, and dynamic-programming-oriented reuse of intermediate information.

The supplied presentation defines the target as a system that accepts tree edges, computes the diameter, displays its length, and highlights the diameter path. The developed project is a web application with a React/TypeScript frontend and an Express/TypeScript backend.

Case Study

A tree is a connected graph with no cycles. For a tree, there is exactly one simple path between any two nodes. The diameter is the maximum number of edges on any such path. Finding the diameter is useful in hierarchical structures, network analysis, routing, and other applications where the greatest distance between connected points is important.

The application works with a generated tree. It accepts a node count from 2 to 1,000 and creates a valid tree with exactly one fewer edge than nodes. It then validates the structure, performs diameter analysis, reconstructs the longest path, and presents the result visually.

Example Scenario

Suppose a tree has several branches from a central region. A path beginning at a leaf on one branch can pass through the central region and finish at a leaf on another branch. If this path contains more edges than every other node-to-node path, it is the tree diameter. The system identifies the two endpoint nodes and the complete sequence of nodes forming the longest path.

The project presentation illustrates a sample result with a diameter of 6 edges. The output includes the diameter length and the corresponding path, which is highlighted in the tree visualization.

System Workflow

STAGE	DESCRIPTION
1. Generate	Select a tree size from 2 to 1,000 nodes.
2. Construct	Create the node and edge representation.
3. Validate	Check size, IDs, edges, duplicates, self-loops, and connectivity.
4. Compute	Build adjacency lists and perform diameter analysis.

5. Visualize	Render the tree using an SVG-based canvas.
6. Highlight	Emphasize diameter nodes and edges and display the result.

Functional Requirements

- Generate a valid tree containing 2–1,000 nodes.
- Construct an adjacency representation.
- Validate the generated tree.
- Compute the diameter length and reconstruct its path.
- Display nodes, edges, diameter, start node, and end node.
- Highlight the diameter path visually.
- Support zoom, panning, fit/reset controls, and dense visualization for larger trees.

Case Study – Implementation Details

Technology and Architecture

Layer	Technology	Purpose
Frontend	React + TypeScript	User interface, navigation, state, and interaction.
Visualization	SVG	Draw nodes/edges and highlight the diameter path.
Backend	Express + TypeScript	Generate, validate, and analyse trees through APIs.
Algorithm	Tree traversal with stored parent/distance state	Find diameter endpoints and reconstruct the path.
Development	VS Code	Write and manage the project source code.

Important Note on the Algorithm Description

The supplied PPT describes the project as using Dynamic Programming. The supplied ZIP implements the diameter calculation through two graph traversals: the first finds a farthest node from an initial node, and the second starts from that farthest node and uses parent information to reconstruct the opposite endpoint's path. The implementation stores and reuses per-node traversal information. This report retains the PPT's Dynamic Programming framing while documenting the supplied implementation accurately.

Handling Up to 1000 Nodes

The project explicitly validates a maximum of 1,000 nodes. A tree with n nodes has $n-1$ edges, and the implementation checks this condition. The visualization adapts spacing for larger trees and uses a dense mode without labels when necessary. Zooming and panning allow larger structures to remain inspectable.

Expected Outcome

The expected outcome is an efficient application that constructs and visualizes a tree, computes its diameter, displays the diameter length and corresponding longest path, and highlights that path on screen. The system is intended to handle tree structures of up to 1,000 nodes while demonstrating the practical use of dynamic-programming concepts and tree traversal techniques.

Input and Output

Input: A generated tree containing N nodes and $N-1$ valid edges.

Processing: Build adjacency lists → validate the tree → perform diameter analysis → reconstruct the path.

Output: Diameter length, start node, end node, ordered diameter path, and an SVG visualization with the diameter highlighted.

ALGORITHM / PSEUDO CODE

Algorithm Objective

The objective is to determine the maximum distance between any two nodes in a tree and return the complete path that produces that distance. The supplied implementation uses adjacency lists and two traversals. The first traversal finds a farthest node from an arbitrary starting node. The second starts from that farthest node, finds the opposite endpoint, stores parent relationships, and reconstructs the path.

Step 1 – Build the Adjacency Structure

Each node receives a list of its neighboring nodes. Since edges are undirected, every edge (u, v) is inserted into both adjacency[u] and adjacency[v].

BUILD-ADJACENCY(tree):

create an empty list for every node

for each edge (u, v) in tree:

add v to adjacency[u]

add u to adjacency[v]

return adjacency

Step 2 – Validate the Tree

The implementation checks the node range, edge count, node IDs, self-loops, duplicate edges, and connectivity.

VALIDATE-TREE(tree):

if nodeCount < 2 or nodeCount > 1000:

return false

if numberOfEdges != nodeCount - 1:

return false

verify node IDs are exactly 1 ... nodeCount

reject self-loops and duplicate edges

traverse from node 1

return true only if every node is visited

Step 3 – First Traversal

Starting from node 1, the traversal records the distance of each discovered node. The node with the greatest distance becomes the first diameter endpoint candidate.

TRAVERSE(adjacency, start):

queue $\leftarrow$ [start]

distance[start] $\leftarrow$ 0

parent $\leftarrow$ empty map

farthest $\leftarrow$ start

while queue is not empty:

current $\leftarrow$ remove front of queue

if distance[current] > distance[farthest]:

farthest $\leftarrow$ current

for each next in adjacency[current]:

if next has no recorded distance:

distance[next] $\leftarrow$ distance[current] + 1

parent[next] $\leftarrow$ current

add next to queue

return farthest, parent

Step 4 – Second Traversal and Path Reconstruction

The second traversal begins at the farthest node found in Step 3. Its farthest node becomes the other endpoint of the diameter. The parent map is followed backwards until the first endpoint is reached, then the collected sequence is reversed.

CALCULATE-DIAMETER(tree):

adjacency $\leftarrow$ BUILD-ADJACENCY(tree)

first $\leftarrow$ TRAVERSE(adjacency, node 1)

second $\leftarrow$ TRAVERSE(adjacency, first.farthest)

path $\leftarrow$ [second.farthest]

cursor $\leftarrow$ second.farthest

while cursor $\neq$ first.farthest:

cursor $\leftarrow$ second.parent[cursor]

append cursor to path

reverse(path)

diameterLength $\leftarrow$ length(path) - 1

return diameter Length, startNode, endNode, path

Step 5 – Visualization

The frontend receives diameter Path from the backend. Consecutive node pairs in this path identify the diameter edges. The Tree Canvas component draws normal edges normally and diameter edges with stronger emphasis. Diameter nodes are also emphasized, while the two endpoints receive additional endpoint styling.

Complexity Analysis

The adjacency representation uses $O(V+E)$ space. Each traversal visits nodes and edges once, giving $O(V+E)$ time. Since a tree has $E=V-1$, this becomes $O(V)$ for each traversal and $O(V)$ overall for the diameter analysis, with $O(V)$ auxiliary storage.

Why This Fits the Project

The approach avoids checking every possible pair of nodes. It uses the tree structure and stored traversal information to identify the two diameter endpoints and reconstruct the full path, which directly supports the project's visual highlighting requirement.

CODE

Core Backend Implementation

The following core module is from the supplied project ZIP and contains the tree model, validation, traversal, and diameter calculation.

```
export type TreeEdge = { source: number; target: number };
export type Tree = {
  nodeCount: number;
  nodes: number[];
  edges: TreeEdge[];
};
export type DiameterAnalysis = {
  diameterLength: number;
  startNode: number;
  endNode: number;
  diameterPath: number[];
  nodeCount: number;
  nodes: number[];
  edges: TreeEdge[];
};
export function generateRandomTree(nodeCount: number): Tree {
  const nodes = Array.from({ length: nodeCount }, (_, index) => index + 1);
  const edges: TreeEdge[] = [];
  for (let node = 2; node <= nodeCount; node += 1) {
    const parent = Math.floor(Math.random() * (node - 1)) + 1;
    edges.push({ source: parent, target: node });
  }
  for (let index = edges.length - 1; index > 0; index -= 1) {
    const swapIndex = Math.floor(Math.random() * (index + 1));
    [edges[index], edges[swapIndex]] = [edges[swapIndex], edges[index]];
  }
}
```

```

}
return { nodeCount, nodes, edges };
}

function buildAdjacency(tree: Tree): Map<number, number[]> {
const adjacency = new Map<number, number[]>();
tree.nodes.forEach((node) => adjacency.set(node, []));
tree.edges.forEach(({ source, target }) => {
adjacency.get(source)?.push(target);
adjacency.get(target)?.push(source);
});
return adjacency;
}

export function isValidTree(tree: Tree): boolean {
if (!Number.isInteger(tree.nodeCount) || tree.nodeCount < 2 || tree.nodeCount > 1000) return false;
if (tree.nodes.length !== tree.nodeCount || tree.edges.length !== tree.nodeCount - 1) return false;

const expectedNodes = new Set(Array.from({ length: tree.nodeCount }, (_, index) => index + 1));

if (new Set(tree.nodes).size !== tree.nodeCount || tree.nodes.some((node) => !expectedNodes.has(node))) return false;

const edgeKeys = new Set<string>();
const adjacency = buildAdjacency(tree);
for (const { source, target } of tree.edges) {
if (source === target || !expectedNodes.has(source) || !expectedNodes.has(target)) return false;

const key = `${Math.min(source, target)}-${Math.max(source, target)}`;
if (edgeKeys.has(key)) return false;
edgeKeys.add(key);
}

const visited = new Set<number>();
const queue = [1];

```

```
while (queue.length) {
  const current = queue.shift();
  if (visited.has(current)) continue;
  visited.add(current);
  adjacency.get(current)?.forEach((neighbor) => {
    if (!visited.has(neighbor)) queue.push(neighbor);
  });
}
return visited.size === tree.nodeCount;
}

function traverse(adjacency: Map<number, number[]>, start: number) {
  const queue = [start];
  const distances = new Map<number, number>([[start, 0]]);
  const parents = new Map<number, number>();
  let farthest = start;
  while (queue.length) {
    const current = queue.shift();
    if ((distances.get(current) ?? 0) > (distances.get(farthest) ?? 0)) farthest = current;
    adjacency.get(current)?.forEach((next) => {
      if (!distances.has(next)) {
        distances.set(next, (distances.get(current) ?? 0) + 1);
        parents.set(next, current);
        queue.push(next);
      }
    });
  }
  return { farthest, parents };
}

export function calculateDiameter(tree: Tree): DiameterAnalysis {
  const adjacency = buildAdjacency(tree);
```

```
const first = traverse(adjacency, tree.nodes[0] ?? 1);
const second = traverse(adjacency, first.farthest);
const path = [second.farthest];
let cursor = second.farthest;
while (cursor !== first.farthest) {
  const parent = second.parents.get(cursor);
  if (parent === undefined) throw new Error("Unable to reconstruct tree diameter path");
  cursor = parent;
  path.push(cursor);
}
const diameterPath = path.reverse();
return {
  diameterLength: diameterPath.length - 1,
  startNode: first.farthest,
  endNode: second.farthest,
  diameterPath,
  nodeCount: tree.nodeCount,
  nodes: tree.nodes,
  edges: tree.edges,
};
}
```

API Route for Diameter Calculation

```
import { Router, type IRouter } from "express";
import {
  CalculateTreeDiameterBody,
  CalculateTreeDiameterResponse,
  GenerateTreeBody,
  GenerateTreeResponse,
} from "@workspace/api-zod";
```

```
import {
  calculateDiameter,
  generateRandomTree,
  isValidTree,
} from "../lib/tree";

const router: IRouter = Router();

router.post("/tree/generate", (req, res): void => {
  const parsed = GenerateTreeBody.safeParse(req.body);
  if (!parsed.success) {
    req.log.warn({ errors: parsed.error.issues }, "Invalid tree generation request");
    res.status(400).json({ error: "Choose a whole number of nodes between 2 and 1,000." });
    return;
  }
  const tree = generateRandomTree(parsed.data.nodes);
  res.json(GenerateTreeResponse.parse(tree));
});

router.post("/tree/diameter", (req, res): void => {
  const parsed = CalculateTreeDiameterBody.safeParse(req.body);
  if (!parsed.success || !isValidTree(parsed.success ? parsed.data : ({}) as never))) {
    req.log.warn("Invalid tree diameter request");
    res.status(400).json({ error: "The generated tree is invalid. Generate a new tree and try again." });
    return;
  }
  try {
    const result = calculateDiameter(parsed.data);
    res.json(CalculateTreeDiameterResponse.parse(result));
  } catch (error) {
    req.log.error({ err: error }, "Tree diameter calculation failed");
  }
}
```

```
res.status(400).json({ error: "The tree diameter could not be calculated. Generate  
a new tree and try again." });
```

```
}
```

```
});
```

```
export default router;
```

RESULTS



